Project Submission

Fundamentals of Web Development
Topic: Library Management Interface

STUDENT NAME	E.Eniya
REGISTER NUMBER	34625U50008
DEPARTMENT	II B.Sc Artificial Intelligence
ID	96EBAA231F7C26BB9664341DAA6C2F72
COLLEGE	M.M.E.S Women's Arts and Science College
DATE OF SUBMISSION	27.10.2026

GITHUB LINK:

1. Problem Statement

Many college and community libraries still keep records of books and issue details in registers or scattered spreadsheets. This makes it slow to search for a book, hard to know whether a book is available, and easy to make mistakes while issuing or returning books. The aim of this project is to design a simple, user-friendly web interface through which a librarian can add books, search the catalogue, issue or return books and see live availability counts without any manual calculation.

2. Objectives

- Provide a clean web interface to manage the library catalogue.
- Allow quick search by book title or author name.
- Show Total, Available and Issued book counts in real time.
- Allow issue, return and delete operations with a single click.
- Build the interface with only HTML, CSS and JavaScript (no framework).

3. System Architecture

Layer	File	Role
Structure	index.html	Page layout: header, stats cards, add-book form, catalogue table
Presentation	style.css	Colours, grid layout, table, buttons, status badges, responsive design
Behaviour	script.js	Book data array, rendering, search, filter, issue/return, add, delete

Flow: Librarian action (add / search / issue / return / delete) → JavaScript updates the **books** array → **renderBooks()** redraws the table → **updateStats()** refreshes the counts.

4. Application Flow

The interface works as a simple cycle: the librarian performs an action, the data is updated, and the screen is redrawn.

1. **Start** - the librarian opens **index.html**. The browser loads **style.css** and **script.js**.
2. **Initial display** - **renderBooks()** draws the default books in the catalogue table and **updateStats()** shows the Total, Available and Issued counts.
3. **Add Book** - the librarian enters title, author and category and clicks Add Book. If any field is empty, the browser asks for it. Otherwise the new book is added to the books array with status Available.
4. **Search / Filter** - typing in the search box or choosing All / Available / Issued selects the matching books using **filter()**.
5. **Issue / Return** - clicking the button on a book calls **toggleIssue()**, which changes the status between Available and Issued.
6. **Delete** - clicking Delete calls **deleteBook()** and removes the book from the array.
7. **Refresh** - after every action the table is redrawn and the counts are updated, so the screen always matches the data.
8. **End** - the cycle repeats until the librarian closes the page.

Application Flowchart

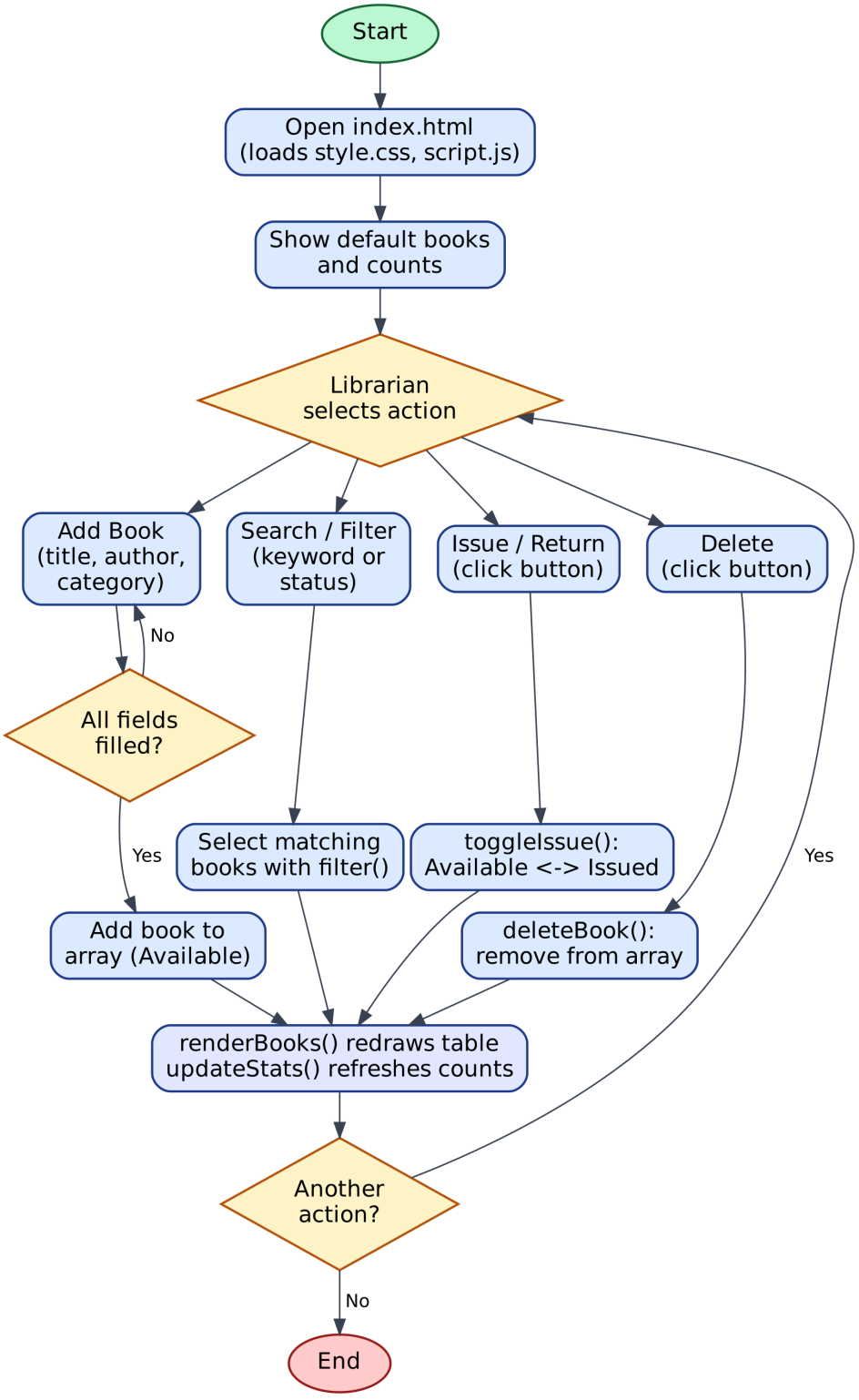


Figure: Application flowchart of the Library Management Interface

5. Features

- **Dashboard cards** - Total Books, Available and Issued counts update instantly.
- **Add New Book** - form with title, author and category fields (all required).
- **Search** - live search by title or author while typing.
- **Filter** - view All, Available only or Issued only books.
- **Issue / Return** - one-click toggle that also changes the status badge colour.
- **Delete** - remove a book from the catalogue.
- **Responsive design** - layout adjusts to mobile screens using a media query.

6. Source Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Library Management Interface</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="topbar">
    <h1>&#128218; City Central Library</h1>
    <p>Library Management Interface</p>
  </header>

  <section class="stats">
    <div class="card"><span id="totalCount">0</span><small>Total Books</small></div>
    <div class="card"><span id="availCount">0</span><small>Available</small></div>
    <div class="card"><span id="issuedCount">0</span><small>Issued</small></div>
  </section>

  <main class="layout">
    <section class="panel">
      <h2>Add New Book</h2>
      <form id="bookForm">
        <input type="text" id="title" placeholder="Book Title" required>
        <input type="text" id="author" placeholder="Author" required>
        <select id="category" required>
          <option value="">Select Category</option>
          <option>Technology</option>
          <option>Science</option>
          <option>Fiction</option>
          <option>History</option>
        </select>
        <button type="submit">Add Book</button>
      </form>
    </section>

    <section class="panel wide">
      <div class="toolbar">
        <h2>Book Catalogue</h2>
        <input type="text" id="search" placeholder="Search by title or author">
        <select id="filter">
          <option value="all">All</option>
          <option value="available">Available</option>
          <option value="issued">Issued</option>
        </select>
      </div>
      <table>
        <thead>
```

```
<tr><th>ID</th><th>Title</th><th>Author</th><th>Category</th><th>Status</th><th>Action</th></tr>
        </thead>
        <tbody id="bookBody"></tbody>
      </table>
    </section>
  </main>

  <footer>&copy; 2026 Library Management Interface | HTML, CSS, JavaScript</footer>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: "Segoe UI", Arial, sans-serif; }
body { background: #eef2f7; color: #1f2937; }

.topbar { background: linear-gradient(90deg, #1e3a8a, #2563eb); color: #fff; text-align: center;
    padding: 22px 10px; }
.topbar h1 { font-size: 28px; }
.topbar p { opacity: .85; margin-top: 4px; }

.stats { display: flex; gap: 16px; justify-content: center; padding: 20px; flex-wrap: wrap; }
.card { background: #fff; border-radius: 10px; padding: 16px 34px; text-align: center; box-shadow: 0
    2px 8px rgba(0,0,0,.08); }
.card span { display: block; font-size: 30px; font-weight: 700; color: #1e3a8a; }
.card small { color: #6b7280; }

.layout { display: grid; grid-template-columns: 300px 1fr; gap: 20px; padding: 0 20px 20px; max-
    width: 1100px; margin: auto; }
.panel { background: #fff; border-radius: 10px; padding: 18px; box-shadow: 0 2px 8px
    rgba(0,0,0,.08); }
.panel h2 { font-size: 18px; margin-bottom: 12px; color: #1e3a8a; }

form { display: flex; flex-direction: column; gap: 10px; }
input, select { padding: 9px 10px; border: 1px solid #cbd5e1; border-radius: 6px; font-size: 14px; }
button { padding: 9px 14px; border: none; border-radius: 6px; background: #2563eb; color: #fff;
    cursor: pointer; font-size: 14px; }
button:hover { background: #1d4ed8; }

.toolbar { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; margin-bottom: 12px; }
.toolbar h2 { margin: 0; flex: 1; }

table { width: 100%; border-collapse: collapse; font-size: 14px; }
th { background: #1e3a8a; color: #fff; text-align: left; padding: 9px; }
td { padding: 9px; border-bottom: 1px solid #e5e7eb; }
tr:hover td { background: #f1f5f9; }

.badge { padding: 3px 10px; border-radius: 12px; font-size: 12px; font-weight: 600; }
.badge.available { background: #dcfce7; color: #166534; }
.badge.issued { background: #fee2e2; color: #991b1b; }
.btn-issue { background: #f59e0b; }
.btn-return { background: #16a34a; }
.btn-delete { background: #dc2626; margin-left: 4px; }

footer { text-align: center; padding: 14px; color: #6b7280; font-size: 13px; }

@media (max-width: 800px) { .layout { grid-template-columns: 1fr; } }
```

script.js

```
// Library Management Interface - script.js
let books = [
  { id: 1, title: "Python Crash Course", author: "Eric Matthes", category: "Technology", issued:
      false },
  { id: 2, title: "Wings of Fire", author: "A.P.J. Abdul Kalam", category: "History", issued: true
      },
  { id: 3, title: "A Brief History of Time", author: "Stephen Hawking", category: "Science", issued:
      false },
  { id: 4, title: "Artificial Intelligence Basics", author: "Tom Taulli", category: "Technology",
      issued: false },
  { id: 5, title: "Ponniyin Selvan", author: "Kalki Krishnamurthy", category: "Fiction", issued:
      true }
];
let nextId = 6;

const bookBody = document.getElementById("bookBody");
const searchBox = document.getElementById("search");
const filterBox = document.getElementById("filter");

function renderBooks() {
  const term = searchBox.value.toLowerCase();
  const filter = filterBox.value;
  bookBody.innerHTML = "";

  books
    .filter(b => b.title.toLowerCase().includes(term) || b.author.toLowerCase().includes(term))
    .filter(b => filter === "all" || (filter === "issued") === b.issued)
    .forEach(b => {
      const row = document.createElement("tr");
      row.innerHTML = `
        <td>${b.id}</td>
        <td>${b.title}</td>
        <td>${b.author}</td>
        <td>${b.category}</td>
        <td><span class="badge ${b.issued ? "issued" : "available"}">${b.issued ? "Issued" :
            "Available"}</span></td>
        <td>
          <button class="${b.issued ? "btn-return" : "btn-issue"}"
              onclick="toggleIssue(${b.id})">${b.issued ? "Return" : "Issue"}</button>
          <button class="btn-delete" onclick="deleteBook(${b.id})">Delete</button>
        </td>`;
      bookBody.appendChild(row);
    });

  updateStats();
}

function updateStats() {
  const issued = books.filter(b => b.issued).length;
  document.getElementById("totalCount").textContent = books.length;
  document.getElementById("issuedCount").textContent = issued;
  document.getElementById("availCount").textContent = books.length - issued;
}

function toggleIssue(id) {
  const book = books.find(b => b.id === id);
  book.issued = !book.issued;
  renderBooks();
}

function deleteBook(id) {
  books = books.filter(b => b.id !== id);
  renderBooks();
}

document.getElementById("bookForm").addEventListener("submit", function (e) {
  e.preventDefault();
  books.push({
    id: nextId++,
```

```
    title: document.getElementById("title").value.trim(),
    author: document.getElementById("author").value.trim(),
    category: document.getElementById("category").value,
    issued: false
  });
  this.reset();
  renderBooks();
});

searchBox.addEventListener("input", renderBooks);
filterBox.addEventListener("change", renderBooks);

renderBooks();
```

7. Output Screenshots

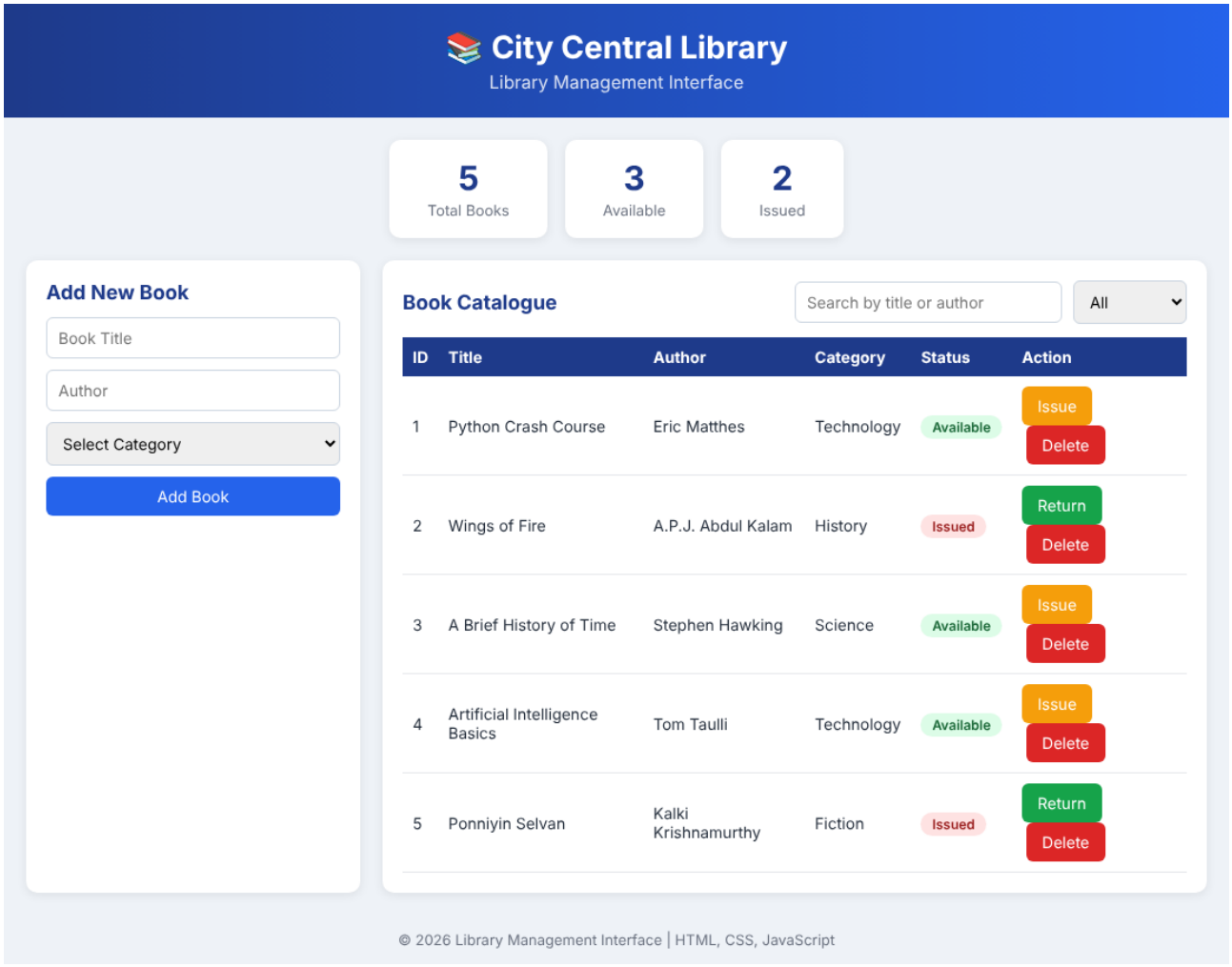


Figure 1: Home screen with dashboard counts and book catalogue

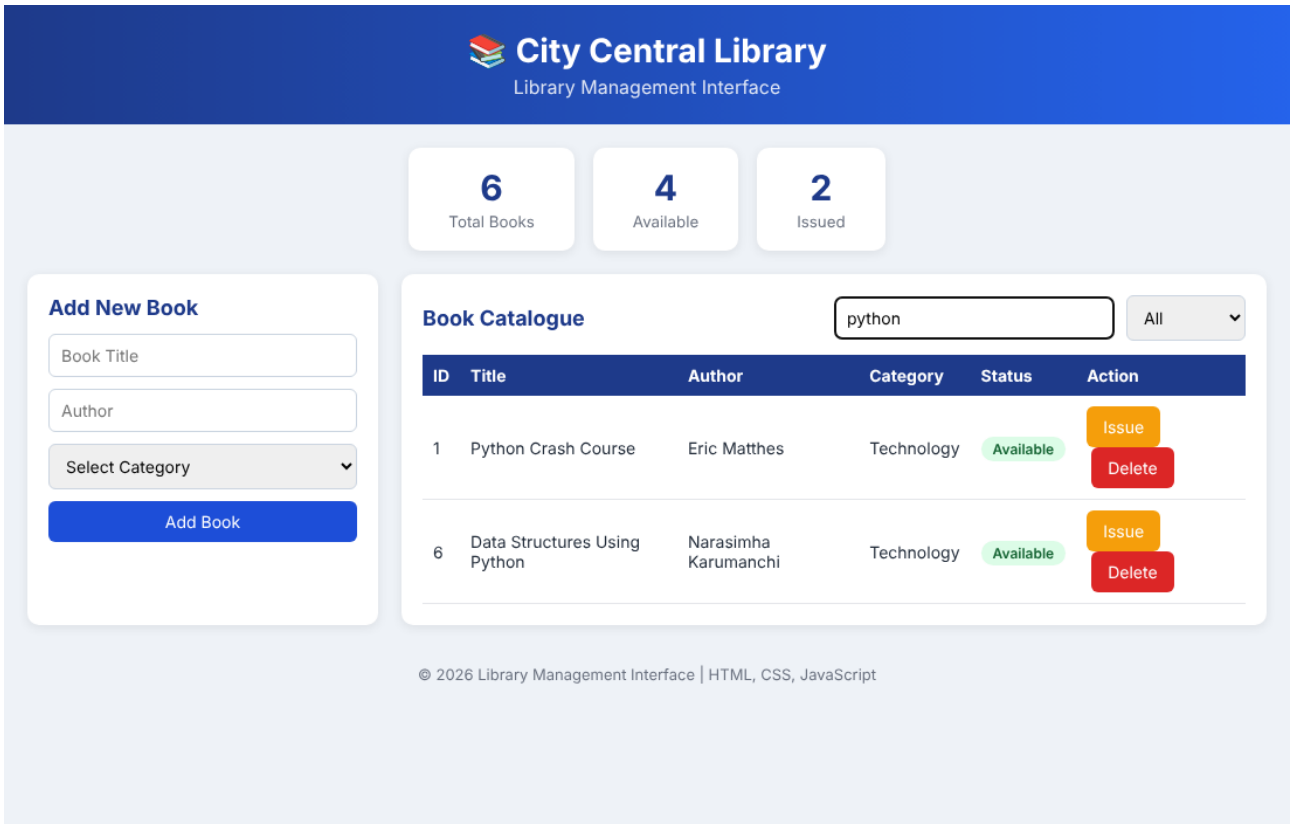


Figure 2: A new book added and the catalogue searched using the keyword "python"

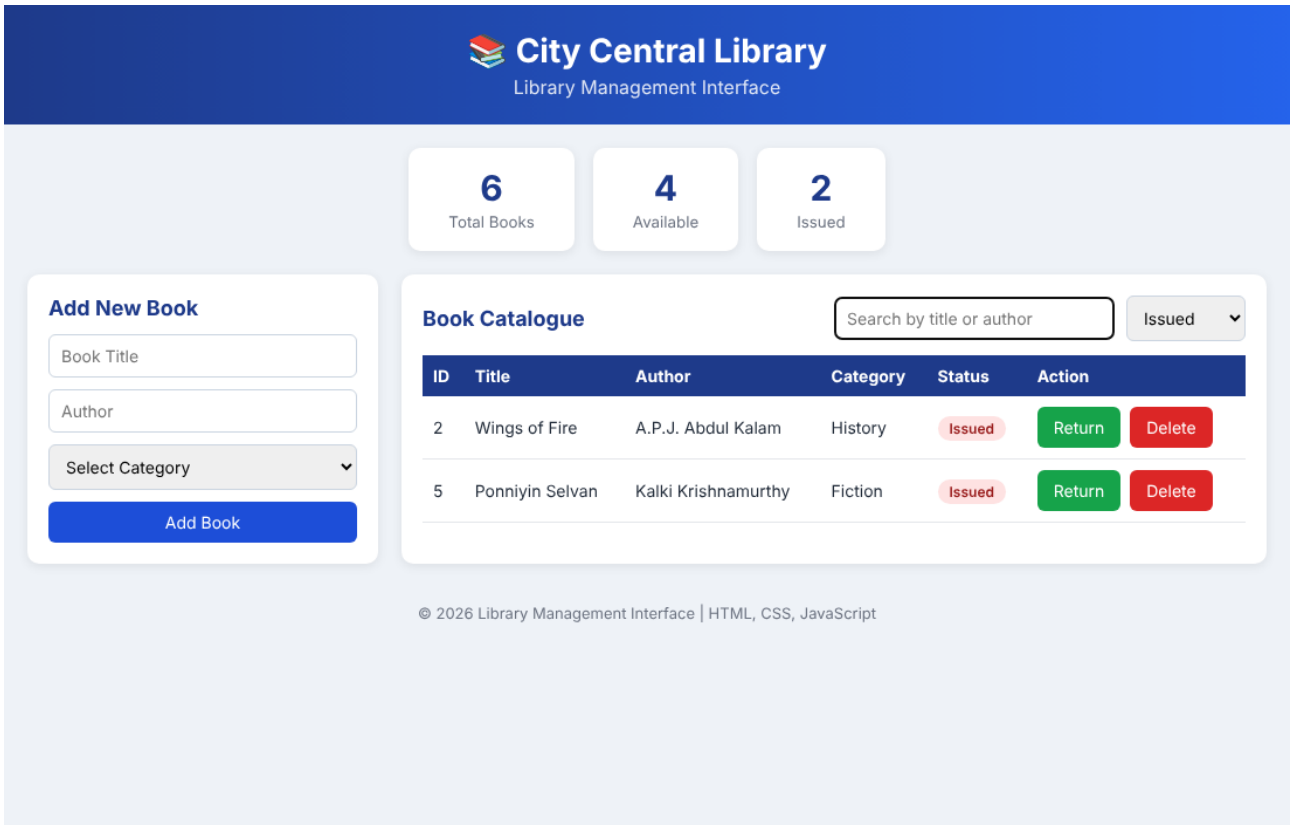


Figure 3: Filter applied to show only issued books

8. Technical Explanation

HTML

index.html defines the page using semantic tags such as header, section, main, form, table and footer. Elements have ids (for example bookBody, search, filter) so that JavaScript can reach them.

CSS

style.css uses CSS Grid for the two-column layout (form on the left, catalogue on the right) and Flexbox for the stat cards and toolbar. Status badges use green for Available and red for Issued. A media query makes the layout single-column on screens narrower than 800px.

JavaScript

script.js stores books as an array of objects (id, title, author, category, issued). renderBooks() reads the search text and filter value, uses filter() to select matching books and builds the table rows. toggleIssue() flips the issued value, deleteBook() removes a book, and the form submit handler pushes a new object into the array. After every change renderBooks() and updateStats() are called so the screen always matches the data.

9. Conclusion

The Library Management Interface shows how a practical, easy-to-use application can be built using only core web technologies. It reduces the effort of searching and tracking books and gives instant information about availability. In future the project can be extended with member login, due dates and fines, local storage or a database, and barcode scanning.